# SIMATS ENGINEERING ASSESSMENT
## TOOL 3

**COURSE NAME: Artificial Intelligence** **COURSE CODE: CSA1708**

---

**COURSE OUTCOME COVERED**

***CO1:*** Apply AI basics such as intelligent agents and uninfor
solve problems effectively. *(BL3)*

*Assessment Tool Weightage: 30%*

---

**Duration : 45 Minutes**

**QUESTIONS: 2** **Total Marks:20**

Annexure A

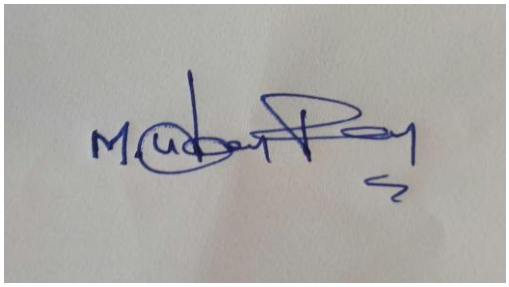
## Descriptive Written Test

---

***Code of Conduct:***

*I M.UDAYKUMAR REDDY (192312588) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***



# Question 1 – Intelligent Agents in a Smart Home Automation System

## 1. Introduction

An intelligent agent is anything that perceives its environment through sensors and acts on it through actuators in order to achieve its objectives. A smart home system is a good example: it senses motion, temperature and time of day, and acts by switching lights, adjusting the thermostat and raising security alerts. Its PEAS description is as follows.

| **PEAS** | **Smart home system** |
|---|---|
| **Performance measure** | Occupant comfort, low energy consumption, security of the home, fast response |
| **Environment** | Rooms, occupants, weather, time of day, doors and windows |
| **Actuators** | Lights, heating/cooling unit, locks, alarm, notifications |
| **Sensors** | Motion detectors, temperature sensors, light sensors, clock, door/window contacts, cameras |

Environment characteristics: partially observable (sensors do not show everything, for example whether a person is still in a room), stochastic (user behaviour and weather are unpredictable), dynamic (conditions change while the agent decides), sequential (current actions affect later energy use and comfort) and continuous (temperature, time).

## 2. Types of intelligent agents and how each works in this system

| **Agent type** | **How it works** | **Smart home example** | **Limitation** |
|---|---|---|---|

| Agent type | How it works | Smart home example | Limitation |
|---|---|---|---|
| **Simple reflex agent** | Chooses an action only from the current percept using condition–action rules (if–then). It has no memory. | IF motion detected AND time = night THEN turn on the light. IF temperature > 30 °C THEN switch on the AC. | Fails when the environment is not fully observable: it cannot tell that someone is sitting still, so it may switch lights off. It cannot learn or plan. |
| **Model-based reflex agent** | Keeps an internal state (a model of how the world works and how its actions change it) built from percept history, and then applies rules to that state. | Remembers that the living room was occupied 2 minutes ago, so the light stays on despite no new motion. Tracks whether windows are open and the temperature trend. | Still reacts by rules only; it has no explicit goal and does not compare alternatives. |
| **Goal-based agent** | Uses the model plus an explicit goal and picks actions through search or planning that lead to the goal. | Goal: room at 24 °C by 7 p.m. when the user returns. The agent plans when to start the AC. Goal: house secured before bedtime, so it plans lock, window and alarm actions. | A goal only says achieved or not achieved. It cannot say which of several ways to reach the goal is better, for example cheaper or more comfortable. |
| **Utility-based agent** | Assigns a numeric utility to each possible outcome and picks the action with the highest (expected) utility, handling trade-offs and uncertainty. | Utility U = w1·Comfort − w2·Energy cost + w3·Security. Pre-cooling at cheap off-peak tariff gives higher utility than cooling at peak price with the same comfort. | Needs a good utility function and more computation. Weights must reflect what the user really values. |
| **Learning agent (extension)** | Has a learning element, a critic, a performance element and a problem generator, and improves from experience. | Learns that the user prefers 22 °C on weekdays and dim lights after 9 p.m., and adjusts its rules and utility weights over time. | Needs time and data. Wrong feedback can teach wrong behaviour. |

## 3. Which agent is most effective? A hybrid approach

No single agent type is enough. Simple reflex agents are fast but forgetful, goal-based agents plan but cannot weigh trade-offs, and only utility-based learning agents can balance comfort, energy and security while adapting to the user. The system also has to learn preferences over time, which requires a learning component. The most effective design is therefore a hybrid, layered agent:

| Layer | Agent type | Role in the system |
|---|---|---|
| **Layer 1 – Safety and emergency** | Simple reflex | Instant response with no deliberation: smoke or intrusion detected → sound alarm, unlock exits, notify the owner. Motion in the dark → light on. |
| **Layer 2 – State tracking** | Model-based | Maintains occupancy, door/window status, temperature trend and the time model, overcoming partial observability. |
| **Layer** | **Agent type** | **Role in the system** |
| **Layer 3 – Planning** | Goal-based | Schedules heating/cooling, lights and lock routines to meet goals such as a comfortable home on arrival and a secured house at night. |
| **Layer 4 – Optimisation** | Utility-based | Chooses between competing actions by maximising U = comfort − energy cost + security (for example, a slightly warmer house to save energy when nobody is home). |
| **Learning component** | Learning agent | Updates preferences and utility weights from user overrides (the critic) so that behaviour improves with time. |

**Justification:** Comfort is achieved by the model-based and learning layers, which know who is home and what they like. Energy efficiency is achieved by the utility layer, which avoids wasteful actions and uses cheap tariff periods. Security is achieved by the reflex layer for immediate response and by the goal-based layer for routines such as locking up. The reflex layer has priority in an emergency, so safety is never delayed by planning.

### 4. Conclusion

A smart home works in a partially observable, changing and stochastic environment and has several competing objectives. A hybrid agent that combines reflex rules for speed, a world model for memory, goals for planning, utility for trade-offs and learning for personalisation gives the best balance of comfort, energy efficiency and security.

## Question 2 – Robot in an Unknown Maze: UCS and IDS (10 Marks)

## 1. Problem formulation

- **States:** positions (cells or junctions) of the maze the robot has discovered.
- **Actions:** move forward, left, right or back, where the path is not blocked by a wall.
- **Step cost:** distance, time or energy of a move; it may be uniform (all 1) or vary (turns, rough floor).
- **Goal test:** the robot is at the exit.
- **Solution:** a sequence of moves from the start to the exit, ideally of minimum cost.

Because the layout is unknown, the robot has no map or heuristic. It must use uninformed (blind) search strategies, which only use the problem definition, and it builds its map from what it senses while moving.

## 2. How UCS and IDS solve the problem

**Uniform Cost Search (UCS).** UCS keeps a priority queue (frontier) ordered by path cost g(n). At every step it removes the node with the lowest cost so far, checks the goal when the node is selected for expansion, and adds its unvisited neighbours with updated costs; a cheaper path to an already-seen node replaces the older one. In the maze, UCS spreads out through the corridors in order of cost and therefore reaches the exit by the cheapest route, even if corridors differ in cost. When all steps cost the same, UCS behaves like Breadth-First Search.

**Iterative Deepening Search (IDS).** IDS runs depth-limited depth-first search with limit 0, then 1, 2, 3 … until the exit is found. Each iteration explores the maze only to the current depth, so it combines the completeness and shortest-path property (in number of moves) of BFS with the small memory of DFS. In a maze with unknown depth of the exit, IDS removes the need to guess a depth limit and avoids getting lost in a very deep wrong corridor.

## 3. Advantages and disadvantages (time and space complexity)

Notation: b = branching factor (choices at a junction), d = depth of the shallowest exit, C* = cost of the optimal solution, ε = minimum step cost.

| **Property** | **Uniform Cost Search** | **Iterative Deepening Search** |
|---|---|---|
| **Strategy** | Expands the lowest-cost node first (priority queue) | Repeated depth-limited DFS with increasing limit |
| **Property** | **Uniform Cost Search** | **Iterative Deepening Search** |
| **Complete?** | Yes, if every step cost $\geq \varepsilon > 0$ | Yes, if b is finite |

| | | |
|---|---|---|
| **Optimal?** | Yes, for any non-negative step costs (minimum total cost) | Yes only if all step costs are equal (shallowest, not cheapest, solution) |
| **Time** | O(b^(1+⌊ C*/ε⌋ )), which can be much larger than b^d when costs are small and varied | O(b^d) |
| **Space** | O(b^(1+⌊ C*/ε⌋ )): it stores the whole frontier | O(b·d): linear in the depth |
| **Advantages** | Finds the cheapest path with varying costs; no depth limit to guess | Very low memory; no depth to guess; complete; optimal for unit costs |
| **Disadvantages** | Explores in all directions, so memory use is very high; slow when many cheap steps exist; no guidance towards the goal | Repeats work at shallow levels; ignores step costs, so it may not return the cheapest path |

**Overhead of IDS:** the repeated expansion looks costly but is small because most nodes are at the deepest level. For b = 3 and d = 4, IDS generates 5 + 4·3 + 3·9 + 2·27 + 81 = 179 nodes, while BFS generates 1 + 3 + 9 + 27 + 81 = 121. That is only about 50% extra work in return for memory of O(bd) instead of O(b^d).

## 4. Relating the problem to types of intelligent agents

| Agent type | Behaviour in the maze | Suitability |
|---|---|---|
| **Simple reflex** | Acts on the current percept, for example the wallfollowing rule “keep the wall on the right”. It has no memory. | Poor: it can loop forever in mazes with cycles. |
| **Model-based** | Keeps a map and the list of visited cells, which removes repeated visits and handles limited sensing. | Necessary: the maze is unknown and partially observable. |
| **Goal-based** | Has the explicit goal “reach the exit” and uses search (UCS/IDS) to plan the moves. | Good: gives purposeful, planned behaviour. |
| **Utility-based** | Chooses the path with the best trade-off between distance, time, energy and risk. | Best when costs vary (UCS naturally fits). |
| **Learning** | Uses earlier runs to improve, for example remembering dead ends in a maze it visits again. | Useful for repeated mazes. |

**How agent design affects decision-making:** the agent's architecture decides what information is used and how. A reflex agent decides from the present percept only, so it is fast but easily trapped. A model-based agent remembers where it has been, so it avoids loops. A goal-based agent compares future outcomes using search, so its decisions are deliberate. A utility-based agent also weighs cost, so its decisions are optimal rather than merely successful. Sensors, memory and the performance measure therefore determine which search strategy can be used.

## 5. Best combination and justification

**Recommended: a model-based, goal-based agent using IDS for exploration together with UCS for cost-optimal planning on the map built so far** (utility-aware).

- **Model-based:** the maze is unknown and only nearby cells are sensed, so the robot must store the discovered map and visited cells to avoid loops.
- **Goal-based:** the task is a clear goal (reach the exit), which search can solve in a planned manner.
- **IDS for exploring:** the depth of the exit is unknown, and IDS needs only O(b·d) memory, which suits a robot with limited memory. It is complete and finds the shallowest exit.
- **UCS for planning:** once parts of the maze are mapped, UCS finds the minimum-cost route when the corridors differ in cost (turns, slopes), which IDS cannot do. If all moves cost the same, IDS alone is sufficient and cheaper in memory.

**Conclusion:** UCS gives optimality and IDS gives memory efficiency. Combined with a model-based, goal-based agent, the robot explores the unknown maze with little memory, remembers what it has seen, and follows the cheapest route to the exit.